

PostgreSQL 適用的 Cloud SQL：開始使用向量嵌入

來源：<https://codelabs.developers.google.com/csql-pg-ai-embedding?hl=zh-tw>

步驟數：13

在本程式碼研究室中，您將瞭解如何使用 Cloud SQL AI 整合功能搭配向量搜尋，並在向量資料上建立索引

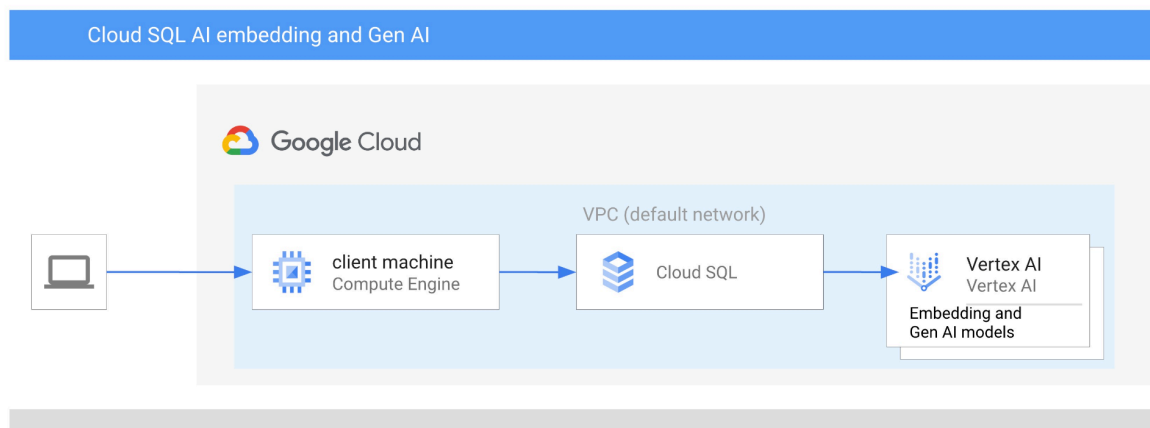
目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 事前準備](#)
4. [4. 建立 Cloud SQL 執行個體](#)
5. [5. 準備資料庫](#)
6. [6. 載入資料](#)
7. [7. 建立嵌入](#)
8. [8. 執行相似度搜尋](#)
9. [9. 使用擷取的資料提升 LLM 回覆品質](#)
10. [10. 建立最鄰近搜尋索引](#)
11. [11. 清理環境](#)
12. [12. 恭喜](#)
13. [13. 問卷調查](#)

步驟 1 · 約 0 分鐘

1. 簡介

在本程式碼實驗室中，您將瞭解如何結合向量搜尋與 Vertex AI 嵌入，使用 PostgreSQL 適用的 Cloud SQL AI 整合功能。



必要條件

- 對 Google Cloud 和控制台有基本瞭解
- 指令列介面和 Cloud Shell 的基本技能

課程內容

- 如何部署 PostgreSQL 適用的 [Cloud SQL](#) 執行個體
- 如何建立資料庫及啟用 [Cloud SQL AI 整合](#)
- 如何將資料載入資料庫
- 如何使用 [Cloud SQL Studio](#)
- 如何在 Cloud SQL 中使用 Vertex AI 嵌入模型
- 如何使用 [Vertex AI Studio](#)
- 如何使用 Vertex AI 生成式模型充實結果
- 如何使用向量索引提升效能

軟硬體需求

- Google Cloud 帳戶和 Google Cloud 專案
 - 支援 Google Cloud 控制台和 Cloud Shell 的網路瀏覽器，例如 [Chrome](#)
-

2. 設定和需求

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

專案設定

1. 登入 [Google Cloud 控制台](#)。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

請改用個人帳戶，而非公司或學校帳戶。

注意：公司和學校帳戶可能設有限制，導致您無法啟用本實驗室所需的 API，或存取部分資源。

2. 建立新專案或重複使用現有專案。如要在 Google Cloud 控制台中建立新專案，請在標題中按一下「選取專案」按鈕，開啟彈出式視窗。



在「選取專案」視窗中，按下「新專案」按鈕，開啟新專案的對話方塊。

Select a project

 NEW PROJECT

 Search projects and folders

在對話方塊中輸入偏好的專案名稱，然後選擇位置。

New Project

Project name *
My Project 13475 ?

Project ID inbound-analogy-389614. It cannot be changed later. [EDIT](#)

Location * [BROWSE](#)

Parent organization or folder

[CREATE](#) [CANCEL](#)

- 專案名稱是這個專案參與者的顯示名稱。專案名稱不會用於 Google API，且隨時可以變更。
- 專案 ID 在所有 Google Cloud 專案中不得重複，且設定後即無法變更。Google Cloud 控制台會自動產生專屬 ID，但您可以自訂 ID。如果不喜歡系統產生的 ID，可以產生另一個隨機 ID，或提供自己的 ID 來檢查是否可用。在大多數程式碼研究室中，您需要參照專案 ID，通常會以 PROJECT_ID 預留位置識別。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用。

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

啟用計費功能

如要啟用計費功能，有兩種方法。你可以使用個人帳單帳戶，也可以按照下列步驟兌換抵免額。

兌換 Google Cloud 抵免額 (選用)

如要參加這個研討會，您需要有具備抵免額的帳單帳戶。請使用本程式碼研究室頂端橫幅中的抵免額開始操作。如果已連結帳單帳戶，可以略過這個步驟。

設定個人帳單帳戶

如果使用 Google Cloud 抵免額設定計費，則可略過此步驟。

如要設定個人帳單帳戶，請[前往這裡在 Cloud 控制台中啟用帳單功能](#)。

注意事項：

- 完成本實驗室的 Cloud 資源費用應不到 \$3 美元。
- 您可以按照本實驗室結尾的步驟刪除資源，以免產生後續費用。
- 新使用者可獲得價值 [\\$300 美元的免費試用期](#)。

啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上工具列的 Cloud Shell 圖示：



你也可以依序按下 G 鍵和 S 鍵。如果您位於 Google Cloud 控制台，或使用這個[連結](#)，這個序列會啟動 Cloud Shell。

佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：

A screenshot of a web browser window showing the Cloud Shell interface. The browser's address bar displays 'qwiklabs-gcp-b3de7b9575cb8d19'. The terminal window has a dark background and shows the following text: 'Welcome to Cloud Shell! Type "help" to get started.' followed by a prompt 'google87764_student@qwiklabs-gcp-b3de7b9575cb8d19:~\$' and a red cursor.

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

步驟 3 · 約 5 分鐘

3. 事前準備

啟用 API

輸出內容：

請注意，啟用部分資源會產生費用。在正常情況下，如果實驗室完成後 Cloud SQL 執行個體和 Compute VM 遭到毀損，所有資源的費用不會超過 \$5 美元。建議您查看帳單，確認這項措施是否可接受。

如要使用 [Cloud SQL](#)、[Compute Engine](#)、[網路服務](#)和 [Vertex AI](#)，您需要在 Google Cloud 雲端專案中啟用各自的 API。

在 Cloud Shell 終端機中，確認專案 ID 已設定完畢：

```
gcloud config set project [YOUR-PROJECT-ID]
```

設定環境變數 PROJECT_ID：

```
PROJECT_ID=$(gcloud config get-value project)
```

啟用所有必要服務：

```
gcloud services enable sqladmin.googleapis.com \
    compute.googleapis.com \
    cloudresourcemanager.googleapis.com \
    servicenetworking.googleapis.com \
    aiplatform.googleapis.com
```

預期的輸出內容：

```
student@cloudshell:~ (test-project-001-402417)$ gcloud config set project test-
project-001-402417
Updated property [core/project].
student@cloudshell:~ (test-project-001-402417)$ PROJECT_ID=$(gcloud config get-value
project)
Your active configuration is: [cloudshell-14650]
student@cloudshell:~ (test-project-001-402417)$
student@cloudshell:~ (test-project-001-402417)$ gcloud services enable
sqladmin.googleapis.com \
    compute.googleapis.com \
    cloudresourcemanager.googleapis.com \
    servicenetworking.googleapis.com \
    aiplatform.googleapis.com
Operation "operations/acat.p2-4470404856-1f44ebd8-894e-4356-bea7-b84165a57442"
finished successfully.
```

API 簡介

- **Cloud SQL Admin API** (sqladmin.googleapis.com) 可讓您以程式輔助方式建立、設定及管理 Cloud SQL 執行個體。這是 Google 全代管關聯式資料庫服務 (支援 MySQL、PostgreSQL 和 SQL Server) 的控制層，可處理佈建、備份、高可用性和資源調度等工作。
 - **Compute Engine API** (compute.googleapis.com) 可讓您建立及管理虛擬機器 (VM)、永久磁碟和網路設定。這個服務提供執行工作負載所需的基礎架構即服務 (IaaS) 基礎，並為許多代管服務代管基礎架構。
 - **Cloud Resource Manager API** (cloudresourcemanager.googleapis.com) 可讓您以程式輔助方式管理 Google Cloud 雲端專案的中繼資料和設定。您可以在專案階層中整理資源、處理 Identity and Access Management (IAM) 政策，以及驗證權限。
 - **Service Networking API** (servicenetworking.googleapis.com) 可讓您自動設定虛擬私有雲 (VPC) 網路與 Google 代管服務之間的私人連線。您必須建立服務 (例如 AlloyDB) 的私人 IP 存取權，才能與其他資源安全地通訊。
 - **Vertex AI API** (aiplatform.googleapis.com) 可讓應用程式建構、部署及擴充機器學習模型。這個平台提供統一的介面，方便您使用所有 Google Cloud AI 服務，包括存取生成式 AI 模型 (如 Gemini) 和模型訓練。
-

步驟 4 · 約 15 分鐘

4. 建立 Cloud SQL 執行個體

建立 Cloud SQL 執行個體，並整合資料庫與 Vertex AI。

建立資料庫密碼

定義預設資料庫使用者的密碼。您可以自行定義密碼，也可以使用隨機函式產生密碼：

```
export CLOUDSQL_PASSWORD=`openssl rand -hex 12`
```

記下系統生成的密碼值：

```
echo $CLOUDSQL_PASSWORD
```

建立 PostgreSQL 適用的 Cloud SQL 執行個體

您可以透過 Google Cloud 控制台、Terraform 等自動化工具或 Google Cloud SDK，以不同方式建立 Cloud SQL 執行個體。在本實驗室中，我們主要會使用 Google Cloud SDK `gcloud` 工具。如要瞭解如何使用其他工具建立執行個體，請參閱 [說明文件](#)。

在 Cloud Shell 工作階段中執行下列指令：

```
gcloud sql instances create my-cloudsql-instance \
--database-version=POSTGRES_17 \
--tier=db-custom-1-3840 \
--region=us-central1 \
--edition=ENTERPRISE \
--enable-google-ml-integration \
--database-flags cloudsql.enable_google_ml_integration=on
```

建立執行個體後，我們需要為執行個體中的預設使用者設定密碼，並確認是否能使用該密碼連線。

```
gcloud sql users set-password postgres \
--instance=my-cloudsql-instance \
--password=$CLOUDSQL_PASSWORD
```

執行方塊中顯示的「`gcloud sql connect`」指令，並在系統提示時輸入密碼，即可連線。

```
gcloud sql connect my-cloudsql-instance --user=postgres
```

使用 `Ctrl+D` 鍵盤快速鍵或執行 `exit` 指令，暫時退出 `psql` 工作階段

```
exit
```

啟用 Vertex AI 整合功能

將必要權限授予內部 Cloud SQL 服務帳戶，以便使用 Vertex AI 整合功能。

找出 Cloud SQL 內部服務帳戶電子郵件地址，並匯出為變數。

```
SERVICE_ACCOUNT_EMAIL=$(gcloud sql instances describe my-cloudsql-instance --  
format="value(serviceAccountEmailAddress)")  
echo $SERVICE_ACCOUNT_EMAIL
```

將 Vertex AI 的存取權授予 Cloud SQL 服務帳戶：

```
PROJECT_ID=$(gcloud config get-value project)  
gcloud projects add-iam-policy-binding $PROJECT_ID \  
--member="serviceAccount:$SERVICE_ACCOUNT_EMAIL" \  
--role="roles/aiplatform.user"
```

如要進一步瞭解如何建立及設定執行個體，請參閱 [這篇文章](#) 中的 Cloud SQL 說明文件。

步驟 5 · 約 5 分鐘

5. 準備資料庫

現在需要建立資料庫並啟用向量支援功能。

建立資料庫

建立名為 *quickstart_db* 的資料庫。您可以透過多種方式建立資料庫，例如使用指令列資料庫用戶端 (如 PostgreSQL 的 psql)、SDK 或 Cloud SQL Studio。我們將使用 SDK (gcloud) 建立資料庫並連線至執行個體。

在 Cloud Shell 中執行指令，建立資料庫

```
gcloud sql databases create quickstart_db --instance=my-cloudsql-instance
```

啟用擴充功能

如要使用 Vertex AI 和向量，我們需要在建立的資料庫中啟用兩個擴充功能。

在 Cloud Shell 中執行指令，連線至建立的資料庫 (您需要提供密碼)

```
gcloud sql connect my-cloudsql-instance --database quickstart_db --user=postgres
```

連線成功後，您需要在 SQL 工作階段中執行兩項指令：

```
CREATE EXTENSION IF NOT EXISTS google_ml_integration CASCADE;  
CREATE EXTENSION IF NOT EXISTS vector CASCADE;
```

結束 SQL 工作階段：

```
exit;
```

6. 載入資料

現在我們需要在資料庫中建立物件並載入資料。我們將使用虛構的 Cymbal 商店資料。資料以 CSV 格式儲存在公開 Google Storage bucket 中。

首先，我們需要在資料庫中建立所有必要物件。為此，我們將使用您已熟悉的 *gcloud sql connect* 和 *gcloud storage* 指令，將結構定義物件下載並匯入資料庫。

在 Cloud Shell 中執行並提供密碼 (建立執行個體時記下的密碼)：

```
gcloud storage cat gs://cloud-training/gcc/gcc-tech-004/cymbal_demo_schema.sql |gcloud sql
connect my-cloudsql-instance --database quickstart_db --user=postgres
```

我們在上一個指令中做了什麼？我們已連線至資料庫，並執行下載的 SQL 程式碼，建立資料表、索引和序列。

下一個步驟是載入資料，為此我們需要從 Google Cloud Storage 下載 CSV 檔案。

```
gcloud storage cp gs://cloud-training/gcc/gcc-tech-004/cymbal_products.csv .
gcloud storage cp gs://cloud-training/gcc/gcc-tech-004/cymbal_inventory.csv .
gcloud storage cp gs://cloud-training/gcc/gcc-tech-004/cymbal_stores.csv .
```

接著，我們需要連線至資料庫。

```
gcloud sql connect my-cloudsql-instance --database quickstart_db --user=postgres
```

並從 CSV 檔案匯入資料。

```
\copy cymbal_products from 'cymbal_products.csv' csv header
```

```
\copy cymbal_inventory from 'cymbal_inventory.csv' csv header
```

```
\copy cymbal_stores from 'cymbal_stores.csv' csv header
```

如果您有自己的資料，且 CSV 檔案與 Cloud 控制台 提供的 Cloud SQL 匯入工具相容，則可使用該工具，不必透過指令列匯入資料。

步驟 7 · 約 5 分鐘

7. 建立嵌入

下一步是使用 Google Vertex AI 的 `textembedding-004` 模型，為產品說明建立嵌入，並將其儲存為向量資料。

連線至資料庫 (如果您已登出或先前的連線已中斷)：

```
gcloud sql connect my-cloudsql-instance --database quickstart_db --user=postgres
```

並使用嵌入函式，在 `cymbal_products` 表格中建立虛擬資料欄 *embedding*。這項指令會建立「*embedding*」虛擬資料欄，用來儲存根據「*product_description*」資料欄生成的嵌入向量。此外，系統也會為資料表中的所有現有資料列建立嵌入內容。模型定義為嵌入函式的第一個參數，來源資料則為第二個參數。

```
ALTER TABLE cymbal_products ADD COLUMN embedding vector(768) GENERATED ALWAYS AS  
(embedding('text-embedding-005',product_description)) STORED;
```

這項作業可能需要一些時間，但 900 到 1000 列的資料通常不會超過 5 分鐘，而且通常會更快。

當我們在資料表中插入新資料列，或更新任何現有資料列的 `product_description` 時，系統會根據「`product_description`」，重新產生「*embedding*」欄的虛擬資料欄資料。

步驟 8 · 約 5 分鐘

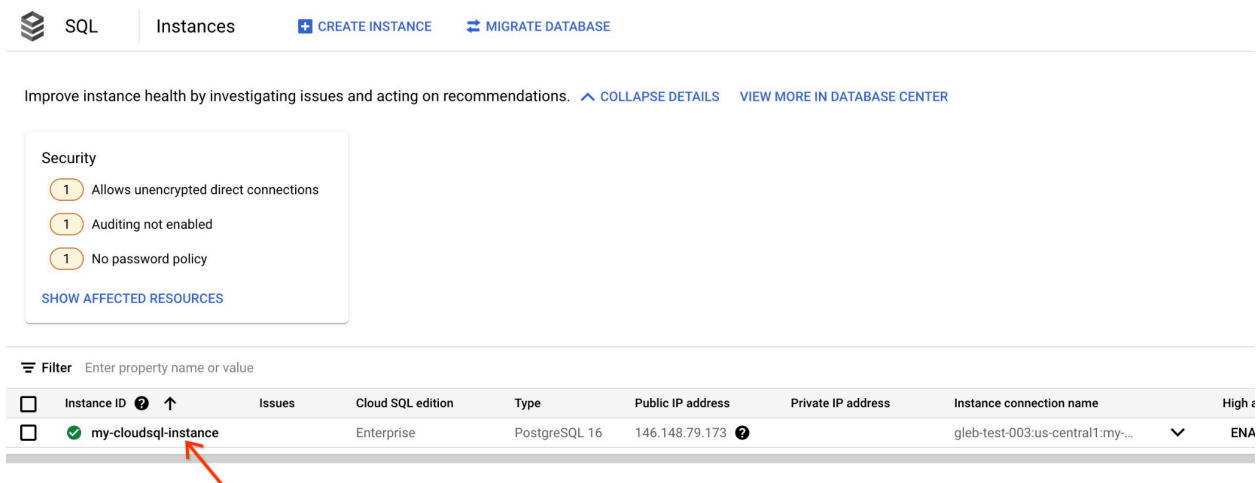
8. 執行相似度搜尋

現在，我們可以根據說明計算出的向量值，以及要求取得的向量值，使用相似度搜尋執行搜尋。

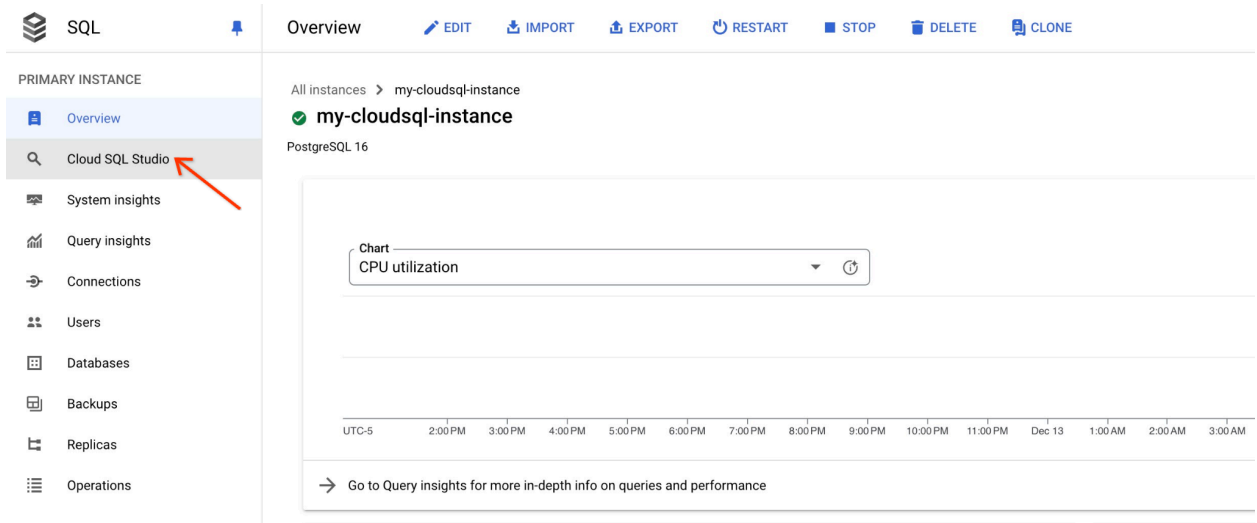
您可以使用 *gcloud sql connect* 從同一個指令列介面執行 SQL 查詢，也可以使用 Cloud SQL Studio 執行。建議在 Cloud SQL Studio 中管理任何多列和複雜查詢。

啟動 Cloud SQL Studio

在**控制台**中，按一下我們稍早建立的 Cloud SQL 執行個體。



開啟後，右側面板會顯示 Cloud SQL Studio。請點按分頁標籤。



系統會開啟對話方塊，供您提供資料庫名稱和憑證：

- 資料庫：quickstart_db
- 使用者：postgres
- 密碼：您記錄的主要資料庫使用者密碼

然後按一下「驗證」按鈕。

All instances > my-cloudsql-instance

PRIM...

Introducing Cloud SQL Studio

Cloud SQL Studio allows you to run queries, create views, modify schemas, and more, all within the console allowing you to quickly experiment with new queries. It g

DISMISS

Sign in to Cloud SQL Studio

Choose a database and user account to securely access your data. [Learn more](#)

Database *
quickstart

Authentication method

☐ IAM database authentication **NEW**

☒ Built-in database authentication

User *
postgres

Password *
.....

[Manage users and their passwords from the \[users page\]\(#\)](#)

AUTHENTICATE

系統會開啟下一個視窗，請按一下右側的「編輯器」分頁標籤，開啟 SQL 編輯器。

All instances > my-cloudsql-instance

PRIM...

Explorer

postgres

- Schema 1
 - public (Default)
 - Tables 0
 - Views 0
 - Functions 0
 - Procedures 0
 - Sequences 0
 - Types 0
 - Extensions 0
 - information_schema
 - pg_catalog

Editor 1

RUN FORMAT CLEAR

1

現在可以執行查詢了。

執行查詢

執行查詢，取得與客戶要求最相關的可用產品清單。我們要傳遞至 Vertex AI 以取得向量值的要求類似於「這裡適合種植哪種果樹？」

以下是您可以執行的查詢，選擇最適合我們要求的 10 個項目：

```

SELECT
    cp.product_name,
    left(cp.product_description,80) as description,
    cp.sale_price,
    cs.zip_code,
    (cp.embedding <=> embedding('text-embedding-005','What kind of fruit trees grow well
here?'))::vector) as distance
FROM
    cymbal_products cp
JOIN cymbal_inventory ci on
    ci.uniq_id=cp.uniq_id
JOIN cymbal_stores cs on
    cs.store_id=ci.store_id
    AND ci.inventory>0
    AND cs.store_id = 1583
ORDER BY
    distance ASC
LIMIT 10;

```

將查詢複製並貼到 Cloud SQL Studio 編輯器，然後按下「執行」按鈕，或將查詢貼到連線至 quickstart_db 資料庫的指令列工作階段。

The screenshot shows the Cloud SQL Studio interface. On the left is the Explorer pane showing the database schema. The main editor shows a SQL query, and the bottom pane displays the results of the query.

Database Explorer:

- quickstart_db
 - Schema 2
 - public (Default)
 - Tables 4
 - cymbal_embedding
 - cymbal_inventory
 - cymbal_products
 - cymbal_stores
 - Views 0
 - Functions 116
 - Procedures 0
 - Sequences 1
 - Types 3
 - Extensions 2
 - google_ml
 - information_schema
 - pg_catalog

SQL Query:

```

1 SELECT
2   cp.product_name,
3   left(cp.product_description,80) as description,
4   cp.sale_price,
5   cs.zip_code,
6   (cp.embedding <=> embedding('text-embedding-004','What kind of fruit trees grow well here?'))::vector) as distance
7 FROM
8   cymbal_products cp
9 JOIN cymbal_inventory ci on
10  ci.uniq_id=cp.uniq_id
11 JOIN cymbal_stores cs on
12  cs.store_id=ci.store_id
13  AND ci.inventory>0
14  AND cs.store_id = 1583
15 ORDER BY
16  distance ASC
17 LIMIT 10;
18

```

RESULTS

product_name	description
Cherry Tree	This is a beautiful cherry tree that will produce delicious cherries. It is an d
Meyer Lemon Tree	Meyer Lemon trees are California's favorite lemon tree! Grow your own lemons by
Toyon	This is a beautiful toyon tree that can grow to be over 20 feet tall. It is an e
California Lilac	This is a beautiful lilac tree that can grow to be over 10 feet tall. It is an d
California Peppertree	This is a beautiful peppertree that can grow to be over 30 feet tall. It is an e
California Black Walnut	This is a beautiful walnut tree that can grow to be over 80 feet tall. It is a d
California Sycamore	This is a beautiful sycamore tree that can grow to be over 100 feet tall. It is
Coast Live Oak	This is a beautiful oak tree that can grow to be over 100 feet tall. It is an ev
Fremont Cottonwood	This is a beautiful cottonwood tree that can grow to be over 100 feet tall. It i

以下是符合查詢條件的產品清單。

product_name	description	sale_price	zip_code	distance
Cherry Tree	This is a beautiful cherry tree that will produce delicious cherries. It is an d	75.00	93230	0.43922018972266397
Meyer Lemon Tree	Meyer Lemon trees are California's favorite lemon tree! Grow your own lemons by	34	93230	0.4685112926118228
Toyon	This is a beautiful toyon tree that can grow to be over 20 feet tall. It is an e	10.00	93230	0.4835677149651668
California Lilac	This is a beautiful lilac tree that can grow to be over 10 feet tall. It is an d	5.00	93230	0.4947204525907498
California Peppertree	This is a beautiful peppertree that can grow to be over 30 feet tall. It is an e	25.00	93230	0.5054166905547247
California Black Walnut	This is a beautiful walnut tree that can grow to be over 80 feet tall. It is a d	100.00	93230	0.5084219510932597
California Sycamore	This is a beautiful sycamore tree that can grow to be over 100 feet tall. It is	300.00	93230	0.5140519790508755
Coast Live Oak	This is a beautiful oak tree that can grow to be over 100 feet tall. It is an ev	500.00	93230	0.5143126438081371
Fremont Cottonwood	This is a beautiful cottonwood tree that can grow to be over 100 feet tall. It i	200.00	93230	0.5174774727252058
Madrone	This is a beautiful madrona tree that can grow to be over 80 feet tall. It is an	50.00	93230	0.5227400803389093

(10 rows)

9. 使用擷取的資料提升 LLM 回覆品質

我們可以運用執行的查詢結果，改善用戶端應用程式的生成式 AI 大型語言模型回覆，並使用提供的查詢結果做為提示的一部分，準備有意義的輸出內容，傳送給 Vertex AI 生成式基礎語言模型。

為此，我們需要使用向量搜尋結果生成 JSON，然後將生成的 JSON 新增至 Vertex AI 中 LLM 模型的提示，藉此建立有意義的輸出內容。首先，我們會生成 JSON，然後在 Vertex AI Studio 中測試，最後將其併入 SQL 陳述式，以便在應用程式中使用。

以 JSON 格式產生輸出內容

修改查詢，以 JSON 格式產生輸出內容，並只傳回一個資料列，以便傳遞至 Vertex AI

PostgreSQL 適用的 Cloud SQL

查詢範例如下：

```
WITH trees as (  
  SELECT  
    cp.product_name,  
    left(cp.product_description,80) as description,  
    cp.sale_price,  
    cs.zip_code,  
    cp.uniq_id as product_id  
  FROM  
    cymbal_products cp  
  JOIN cymbal_inventory ci on  
    ci.uniq_id=cp.uniq_id  
  JOIN cymbal_stores cs on  
    cs.store_id=ci.store_id  
    AND ci.inventory>0  
    AND cs.store_id = 1583  
  ORDER BY  
    (cp.embedding <=> embedding('text-embedding-005','What kind of fruit trees grow well  
here?'))::vector) ASC  
  LIMIT 1)  
SELECT json_agg(trees) FROM trees;
```

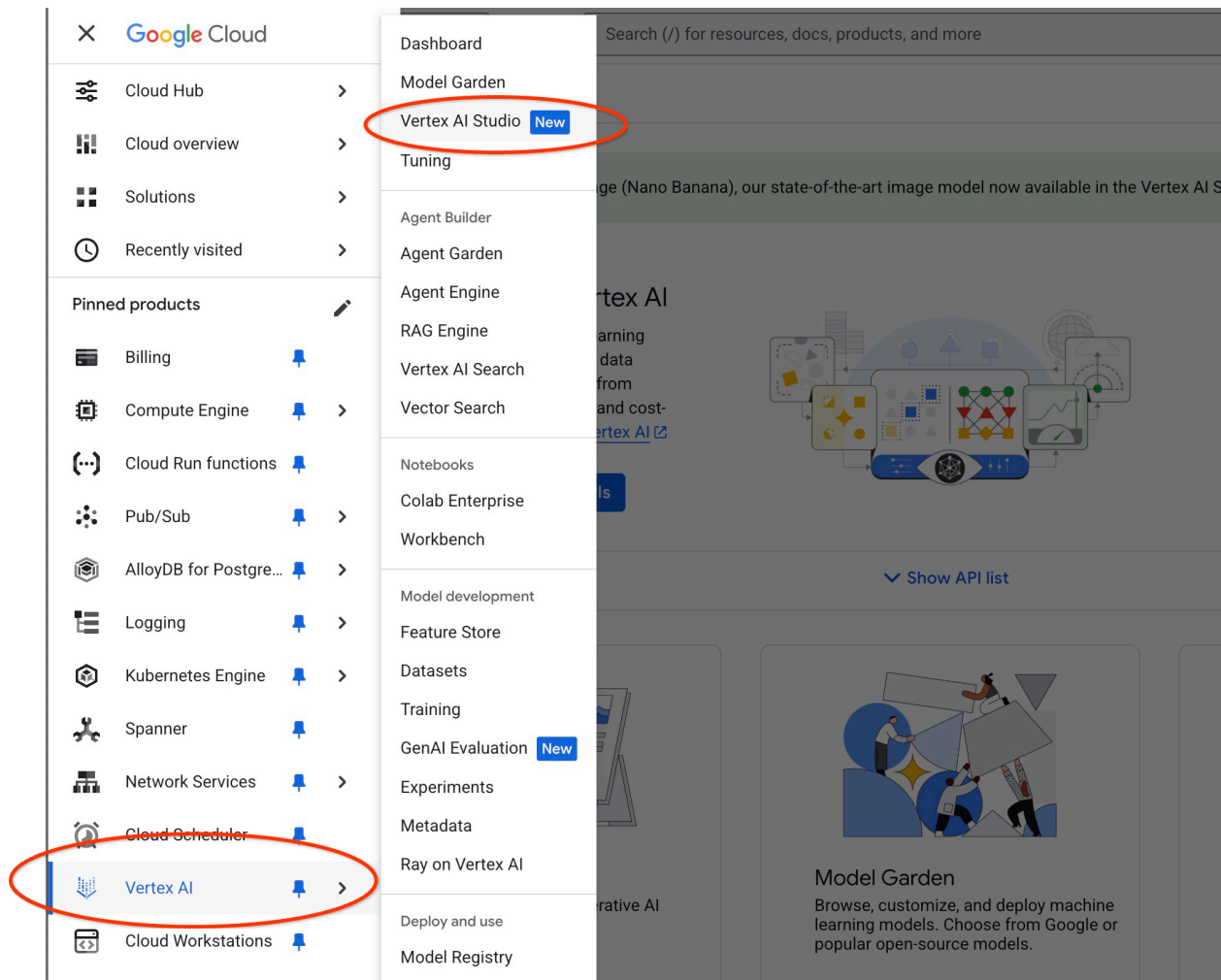
以下是預期的 JSON 輸出內容：

```
[{"product_name":"Cherry Tree","description":"This is a beautiful cherry tree that  
will produce delicious cherries. It is an  
d","sale_price":75.00,"zip_code":93230,"product_id":"d536e9e823296a2eba198e52dd23e712  
"}]
```

在 Vertex AI Studio 中執行提示

我們可以將產生的 JSON 做為提示詞的一部分，提供給 Vertex AI Studio 中的生成式 AI 文字模型

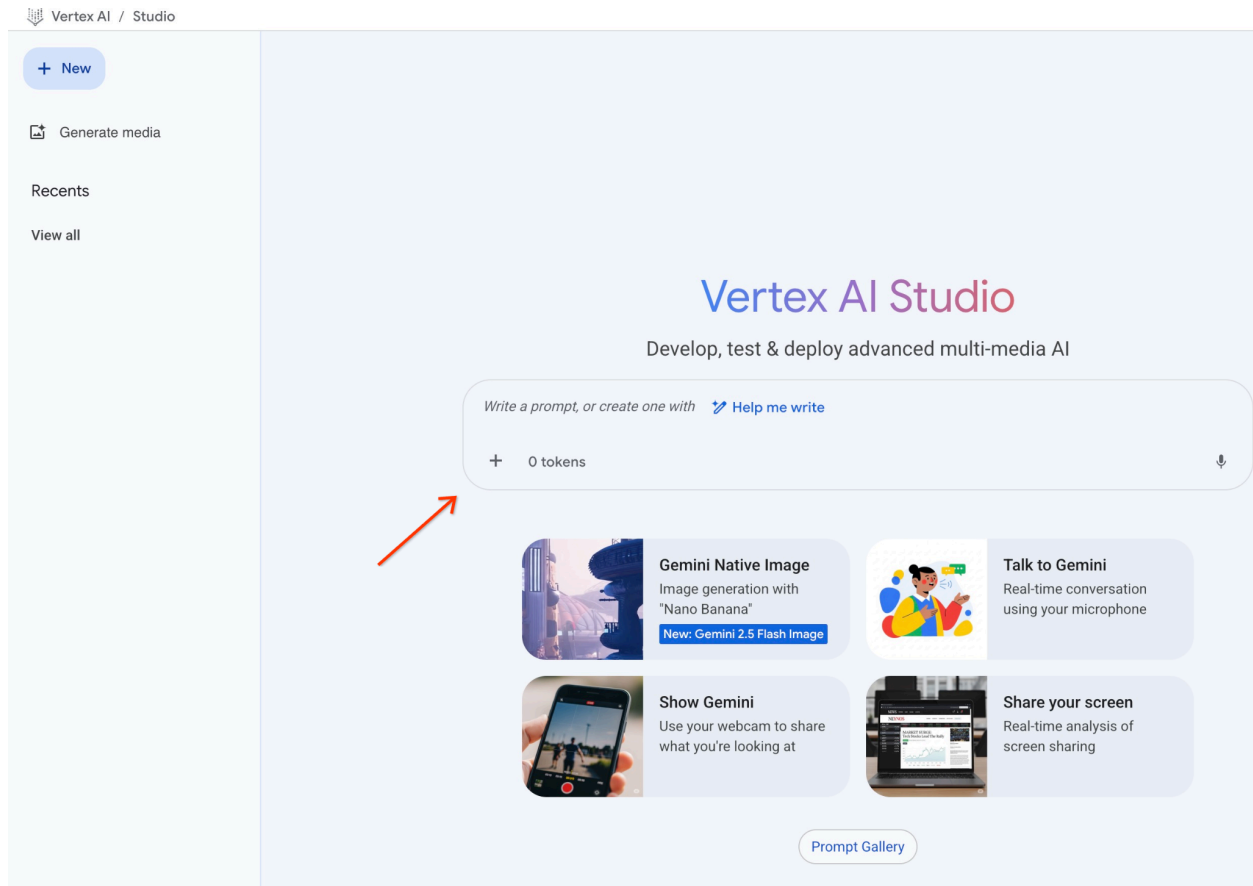
在 Cloud 控制台中開啟 Vertex AI Studio。



注意：在範例中，我們使用 Vertex Studio 的所有預設參數，以及當時最新的 gemini-3-pro-preview 模型和預設參數。輸出結果可能因模型版本和參數而異。請參閱[說明文件](#)，進一步瞭解 Vertex AI 和生成式語言模型。

系統可能會要求您啟用其他 API，但您可以忽略這項要求。我們不需要任何其他 API 即可完成實驗室。

在 Studio 中輸入提示。



我們將使用以下提示：

```
You are a friendly advisor helping to find a product based on the customer's needs.
Based on the client request we have loaded a list of products closely related to search.
The list in JSON format with list of values like {"product_name":"name","description":"some
description","sale_price":10,"zip_code": 10234, "produt_id":
"02056727942aeb714dc9a2313654e1b0"}
Here is the list of products:
[place your JSON here]
The customer asked "What tree is growing the best here?"
You should give information about the product, price and some supplemental information.
Do not ask any additional questions and assume location based on the zip code provided in the
list of products.
```

以下是將 JSON 預留位置替換為查詢回應後的樣子：

You are a friendly advisor helping to find a product based on the customer's needs. Based on the client request we have loaded a list of products closely related to search.

The list in JSON format with list of values like

```
{ "product_name": "name", "description": "some description", "sale_price": 10, "zip_code": 10234, "product_id": "02056727942aeb714dc9a2313654e1b0" }
```

Here is the list of products:

```
[ { "product_name": "Cherry Tree", "description": "This is a beautiful cherry tree that will produce delicious cherries. It is an d", "sale_price": 75.00, "zip_code": 93230, "product_id": "d536e9e823296a2eba198e52dd23e712" } ]
```

The customer asked "What tree is growing the best here?"

You should give information about the product, price and some supplemental information.

Do not ask any additional questions and assume location based on the zip code provided in the list of products.

以下是使用 JSON 值執行提示時的結果：

Vertex AI / Studio / Chat

New chat

Generate media

You are a friendly advisor helping to find a product based on the customer's needs. Based on the client request we have loaded a list of products closely related to search. The list in JSON format with list of values like { "product_name": "name", "description": "some description", "sale_price": 10, "zip_code": 10234, "product_id": "02056727942aeb714dc9a2313654e1b0" } Here is the list of products: [{ "product_name": "Cherry Tree", "description": "This is a beautiful cherry tree that will produce delicious cherries. It is an d", "sale_price": 75.00, "zip_code": 93230, "product_id": "d536e9e823296a2eba198e52dd23e712" }] The customer asked "What tree is growing the best here?" You should give information about the product, price and some supplemental information. Do not ask any additional questions and assume location based on the zip code provided in the list of products.

Thought for 10 seconds

Based on the zip code provided in our product list (93230), it looks like you are located in the **Hanford, California** area. 1 3 4 6 This region, located in the San Joaquin Valley, is actually one of the premier locations in the country for growing stone fruits.

To answer your question, the best tree growing in your area from our selection is the **Cherry Tree**.

Here are the details for you:

- **Product:** Cherry Tree
- **Price:** \$75.00
- **Description:** This is a beautiful cherry tree that will produce delicious cherries. 2

Why it grows best here: The San Joaquin Valley climate offers the specific conditions cherries need to thrive: long, warm summer days for fruit development and cool nights. 5 Because Hanford is in a major agricultural hub known for its stone fruit production, you can expect this tree to be very productive in your local soil. Just be sure to plant it in a spot with full sun and well-drained soil to get the best harvest

Grounding Sources 2

1. zip-codes.com
2. anawaltlumber.com
3. unitedstateszipcodes.org
4. zipdatamaps.com
5. ucdavis.edu
6. greatdata.com

Grounding Sources

本範例中，模型回覆的答案如下。請注意，由於模型和參數會隨時間變更，因此您收到的答案可能有所不同：

「根據產品清單中提供的郵遞區號 (93230)，你似乎位於加州漢福特地區。1346 這個位於聖華金谷的地區，實際上是美國種植核果的主要地點之一。

根據我們的選擇，最適合您所在地區的靈樹是櫻桃樹。

詳情如下：

產品：櫻桃樹

價格：\$75.00 美元

說明：這是一棵美麗的櫻桃樹，會結出美味的櫻桃。2

為何這裡最適合種植：聖華金谷的氣候提供櫻桃生長所需的特定條件：漫長溫暖的夏季有助於果實發育，涼爽的夜晚則能促進生長。5 漢福德是主要的農業中心，以生產核果聞名，因此您可期待這棵樹在當地土壤中結實累累。只要確保種植在日照充足且排水良好的土壤中，就能獲得豐收」

在 PSQL 中執行提示

我們也可以使用 Cloud SQL AI 與 Vertex AI 的整合功能，直接在資料庫中透過 SQL，從生成模型取得類似的回覆。但如要使用 gemini-2.0-flash-exp 模型，我們需要先註冊。

在 PostgreSQL 適用的 Cloud SQL 中執行

如果目前版本較舊，請將擴充功能升級至 1.4.2 以上版本。如先前所示，從 *gcloud sql connect* 連線至 `quickstart_db` 資料庫 (或使用 Cloud SQL Studio)，然後執行：

```
SELECT extversion from pg_extension where extname='google_ml_integration';
```

如果傳回的值小於 1.4.3，請執行下列指令：

```
ALTER EXTENSION google_ml_integration UPDATE TO '1.4.3';
```

接著，我們需要將 `google_ml_integration.enable_model_support` 資料庫旗標設為「on」。如要驗證目前的設定，請執行。

```
show google_ml_integration.enable_model_support;
```

psql 工作階段的預期輸出內容為「on」：

```
quickstart_db => show google_ml_integration.enable_model_support;
google_ml_integration.enable_model_support
-----
on
(1 row)
```

如果顯示「off」，則需要更新資料庫旗標。如要執行這項操作，可以使用網頁控制台介面，或執行下列 *gcloud* 指令。

```
gcloud sql instances patch my-cloudsql-instance \
--database-flags
google_ml_integration.enable_model_support=on,cloudsql.enable_google_ml_integration=on
```

指令會在背景執行，大約需要 1 到 3 分鐘。接著，您可以在 psql 工作階段中驗證新旗標，也可以使用 Cloud SQL Studio 連線至 `quickstart_db` 資料庫。

```
show google_ml_integration.enable_model_support;
```

psql 工作階段的預期輸出內容為「on」：

```
quickstart_db => show google_ml_integration.enable_model_support;
google_ml_integration.enable_model_support
-----
on
(1 row)
```

接著，我們需要註冊兩個模型。第一個是已使用的 *text-embedding-005* 模型。由於我們已啟用模型註冊功能，因此需要註冊。

如要在 psql 或 Cloud SQL Studio 中註冊模型執行作業，請執行下列程式碼：

```
CALL
google_ml.create_model(
  model_id => 'text-embedding-005',
  model_provider => 'google',
  model_qualified_name => 'text-embedding-005',
  model_type => 'text_embedding',
  model_auth_type => 'cloudsql_service_agent_iam',
  model_in_transform_fn => 'google_ml.vertexai_text_embedding_input_transform',
  model_out_transform_fn => 'google_ml.vertexai_text_embedding_output_transform');
```

我們需要註冊的下一個模型是 *gemini-2.0-flash-001*，這個模型會用於生成易於理解的輸出內容。

請在 *model_request_url* 字串值中，將預留位置 *\$PROJECT_ID* 替換為您的專案 ID。您也可以選擇在執行下列指令前，更正模型名稱，改用更進階的模型。

```
CALL
google_ml.create_model(
  model_id => 'gemini-2.5-flash',
  model_request_url => 'https://us-central1-
aiplatform.googleapis.com/v1/projects/$PROJECT_ID/locations/us-
central1/publishers/google/models/gemini-2.5-flash:streamGenerateContent',
  model_provider => 'google',
  model_auth_type => 'cloudsql_service_agent_iam');
```

您隨時可以從 *google_ml.model_info_view* 中選取資訊，驗證已註冊的模型清單。

```
select model_id,model_type from google_ml.model_info_view;
```

以下是輸出內容範例

```
quickstart_db=> select model_id,model_type from google_ml.model_info_view;
```

model_id	model_type
-----+-----	
textembedding-gecko	text_embedding
textembedding-gecko@001	text_embedding
gemini-1.5-pro:streamGenerateContent	generic
gemini-1.5-pro:generateContent	generic
gemini-1.0-pro:generateContent	generic
text-embedding-005	text_embedding
gemini-2.5-flash	generic

現在，我們可以使用子查詢 JSON 中產生的內容，透過 SQL 將其提供給生成式 AI 文字模型，做為提示詞的一部分。

在資料庫的 psql 或 Cloud SQL Studio 工作階段中執行查詢

```

WITH trees AS (
SELECT
    cp.product_name,
    cp.product_description AS description,
    cp.sale_price,
    cs.zip_code,
    cp.uniq_id AS product_id
FROM
    cymbal_products cp
JOIN cymbal_inventory ci ON
    ci.uniq_id = cp.uniq_id
JOIN cymbal_stores cs ON
    cs.store_id = ci.store_id
    AND ci.inventory>0
    AND cs.store_id = 1583
ORDER BY
    (cp.embedding <=> google_ml.embedding('text-embedding-005',
    'What kind of fruit trees grow well here?')::vector) ASC
LIMIT 1),
prompt AS (
SELECT
    'You are a friendly advisor helping to find a product based on the customer''s needs.
Based on the client request we have loaded a list of products closely related to search.
The list in JSON format with list of values like
{"product_name":"name","product_description":"some description","sale_price":10}
Here is the list of products:' || json_agg(trees) || 'The customer asked "What kind of fruit
trees grow well here?"
You should give information about the product, price and some supplemental information' AS
prompt_text
FROM
    trees),
response AS (
SELECT
    json_array_elements(google_ml.predict_row( model_id =>'gemini-2.5-flash',
request_body => json_build_object('contents',
json_build_object('role',
'user',
'parts',
json_build_object('text',
prompt_text))))->'candidates'->0->'content'->'parts'->0->'text' AS resp
FROM
    prompt)
SELECT
    string_agg(resp::text,
    ' ')
FROM
    response;

```

預期輸出內容如下。輸出內容可能因模型版本和參數而異：

"That's a great question! It sounds like you're looking to add some delicious fruit to your garden.\n\nBased on the products we have that are closely related to your search, I can tell you about a fantastic option:\n\n**Cherry Tree**" "\n\n**Description:** This beautiful deciduous tree will produce delicious cherries. It grows to be about 15 feet tall, with dark green leaves in summer that turn a beautiful red in the fall. Cherry trees are known for their beauty, shade, and privacy. They prefer a cool, moist climate and sandy soil." "\n\n**Price:** \$75.00\n\n**Grows well in:** USDA Zones 4-9.\n\nTo confirm if this Cherry Tree will thrive in your specific location, you might want to check which USDA Hardiness Zone your area falls into. If you're in zones 4-9, this" " could be a wonderful addition to your yard!"

步驟 10 · 約 5 分鐘

10. 建立最鄰近搜尋索引

我們的資料集相當小，回覆時間主要取決於與 AI 模型的互動。但如果向量數量達到數百萬，向量搜尋可能會佔用大量回應時間，並對系統造成高負載。為改善這點，我們可以在向量上建立索引。

建立 HNSW 索引

我們將在測試中使用 HNSW 索引類型。HNSW 代表*階層式可導覽小世界*，是多層圖形索引。

如要為嵌入資料欄建立索引，我們需要定義嵌入資料欄、距離函式，以及 m 或 ef_constructions 等選用參數。如要進一步瞭解參數，請參閱[說明文件](#)。

```
CREATE INDEX cymbal_products_embeddings_hnsw ON cymbal_products
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
```

預期輸出內容：

```
quickstart_db=> CREATE INDEX cymbal_products_embeddings_hnsw ON cymbal_products
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
CREATE INDEX
quickstart_db=>
```

比較回覆

現在我們可以在 *EXPLAIN* 模式下執行向量搜尋查詢，並驗證是否已使用索引。

```
EXPLAIN (analyze)
WITH trees as (
SELECT
    cp.product_name,
    left(cp.product_description,80) as description,
    cp.sale_price,
    cs.zip_code,
    cp.uniq_id as product_id
FROM
    cymbal_products cp
JOIN cymbal_inventory ci on
    ci.uniq_id=cp.uniq_id
JOIN cymbal_stores cs on
    cs.store_id=ci.store_id
    AND ci.inventory>0
    AND cs.store_id = 1583
ORDER BY
    (cp.embedding <=> embedding('text-embedding-005','What kind of fruit trees grow well
here?'))::vector) ASC
LIMIT 1)
SELECT json_agg(trees) FROM trees;
```

預期輸出內容：

```

Aggregate (cost=779.12..779.13 rows=1 width=32) (actual time=1.066..1.069 rows=1
loops=1)
  -> Subquery Scan on trees (cost=769.05..779.12 rows=1 width=142) (actual
time=1.038..1.041 rows=1 loops=1)
    -> Limit (cost=769.05..779.11 rows=1 width=158) (actual time=1.022..1.024
rows=1 loops=1)
      -> Nested Loop (cost=769.05..9339.69 rows=852 width=158) (actual
time=1.020..1.021 rows=1 loops=1)
        -> Nested Loop (cost=768.77..9316.48 rows=852 width=945)
(actual time=0.858..0.859 rows=1 loops=1)
          -> Index Scan using cymbal_products_embeddings_hnsw on
cymbal_products cp (cost=768.34..2572.47 rows=941 width=941) (actual
time=0.532..0.539 rows=3 loops=1)
            Order By: (embedding <=>
'[0.008864171,0.03693164,-0.024245683,...
<redacted>
...,0.017593635,-0.040275685,-0.03914233,-0.018452475,0.00826032,-0.07372604
]':::vector)
          -> Index Scan using product_inventory_pkey on
cymbal_inventory ci (cost=0.42..7.17 rows=1 width=37) (actual time=0.104..0.104
rows=0 loops=3)
            Index Cond: ((store_id = 1583) AND (uniq_id =
(cp.uniq_id)::text))
            Filter: (inventory > 0)
            Rows Removed by Filter: 1
          -> Materialize (cost=0.28..8.31 rows=1 width=8) (actual
time=0.133..0.134 rows=1 loops=1)
            -> Index Scan using product_stores_pkey on cymbal_stores
cs (cost=0.28..8.30 rows=1 width=8) (actual time=0.129..0.129 rows=1 loops=1)
              Index Cond: (store_id = 1583)

Planning Time: 112.398 ms
Execution Time: 1.221 ms

```

從輸出內容中，我們清楚看到查詢使用「Index Scan using cymbal_products_embeddings_hnsw」。

如果我們執行查詢時沒有說明：

```

WITH trees as (
SELECT
    cp.product_name,
    left(cp.product_description,80) as description,
    cp.sale_price,
    cs.zip_code,
    cp.uniq_id as product_id
FROM
    cymbal_products cp
JOIN cymbal_inventory ci on
    ci.uniq_id=cp.uniq_id
JOIN cymbal_stores cs on
    cs.store_id=ci.store_id
    AND ci.inventory>0
    AND cs.store_id = 1583
ORDER BY
    (cp.embedding <=> embedding('text-embedding-005','What kind of fruit trees grow well
here?'))::vector) ASC
LIMIT 1)
SELECT json_agg(trees) FROM trees;

```

預期輸出內容 (輸出內容可能因模型和索引而異)：

```

[{"product_name":"Cherry Tree","description":"This is a beautiful cherry tree that
will produce delicious cherries. It is an
d","sale_price":75.00,"zip_code":93230,"product_id":"d536e9e823296a2eba198e52dd23e712
"}]

```

我們可以發現結果相同，並傳回與未建立索引時搜尋結果頂端相同的櫻桃樹。視參數和索引類型而定，結果可能會略有不同，並傳回樹狀結構的不同頂端記錄。在測試期間，有索引的查詢在 131.301 毫秒內傳回結果，沒有索引的查詢則在 167.631 毫秒內傳回結果，但我們處理的是非常小的資料集，如果資料量更大，差異會更顯著。

您可以試用向量可用的不同索引，並在[說明文件](#)中查看更多實驗室和 LangChain 整合範例。

11. 清理環境

刪除 Cloud SQL 執行個體

完成實驗室後，請終止 Cloud SQL 執行個體

如果連線中斷，且所有先前的設定都遺失，請在 Cloud Shell 中定義專案和環境變數：

```
export INSTANCE_NAME=my-cloudsql-instance
export PROJECT_ID=$(gcloud config get-value project)
```

刪除執行個體：

```
gcloud sql instances delete $INSTANCE_NAME --project=$PROJECT_ID
```

注意：這個指令需要 3 到 5 分鐘才能執行完畢

預期的控制台輸出內容：

```
student@cloudshell:~$ gcloud sql instances delete $INSTANCE_NAME --
project=$PROJECT_ID
All of the instance data will be lost when the instance is deleted.

Do you want to continue (Y/n)? y

Deleting Cloud SQL instance...done.
Deleted [https://sandbox.googleapis.com/v1beta4/projects/test-project-001-
402417/instances/my-cloudsql-instance].
```

12. 恭喜

恭喜您完成本程式碼研究室。

這個實驗室屬於「Google Cloud 學習路徑：打造可用於正式環境的 AI」。

- [探索完整課程](#)，從設計原型開始，一步步把專案投入正式環境。
- 使用主題標記 `#ProductionReadyAI` 分享你的進度。

涵蓋內容

- 如何部署 PostgreSQL 適用的 [Cloud SQL](#) 執行個體
- 如何建立資料庫及啟用 [Cloud SQL AI 整合](#)
- 如何將資料載入資料庫
- 如何使用 [Cloud SQL Studio](#)
- 如何在 Cloud SQL 中使用 Vertex AI 嵌入模型
- 如何使用 [Vertex AI Studio](#)
- 如何使用 Vertex AI 生成式模型充實結果
- 如何使用向量索引提升效能

請嘗試使用 ScaNN 索引 (而非 HNSW) 進行類似的 [AlloyDB 程式碼研究室](#)

步驟 13 · 約 1 分鐘

13. 問卷調查

輸出內容：
